

AGORA

Development Log

*A plain record of how the market simulator was built,
what was added, what was taken out, and why.*

Investment portfolio . Quantitative project . August to September 2026

1. What this is

Agora is a stock market simulator I built for my investment portfolio. Instead of drawing a price from a formula, it fills a market with lots of small software traders, each following one simple rule, and lets them buy and sell from each other. The price is just whatever they last traded at. Realistic behaviour appears from the crowd, not from an equation.

This document is a log. It records what I built, in the order I built it, what I added, what I took out, and why. It is honest about the parts that did not work and what I changed.

A note on the coding

I am not proficient at coding. The code was written with the help of AI. I set the direction, made the decisions, judged the results, and tested the app. The AI did the typing and the technical detail. I have tried to understand each part, and near the end I had the whole thing rebuilt in a simpler way so that I can actually follow it.

2. The plan

I started from a design document. It set out the idea, the kinds of traders, the order book, and the checks that would tell me whether the market looked real. The plan was ambitious. The build did not follow it exactly, which is normal. This log is what actually happened.

3. The first engine

The first version had an order book and four core traders. Noise traders for random flow. A market maker to quote both sides and earn the spread. Momentum traders that buy what is rising. Value investors that buy cheap and sell dear against a fair value. I got it running and it produced a price.

The first result was wrong. The price ran away to thirty thousand, from a start of one hundred. The momentum traders were bidding it up and nothing was strong enough to pull it back. I fixed it three ways. I made the market maker's spread move with volatility and stay bounded. I made the value investors patient, resting orders near fair value instead of chasing. And I calmed the momentum traders. After that the price stayed in a sensible range.

4. Checking it looks real

A simulation is only useful if it behaves like a real market. I added two checks. Fat tails, which means big moves happen far more often than a bell curve says. Volatility clustering, which means busy and calm periods come in runs.

The first time, the checks failed. I learned that these features live in fast, minute by minute returns and fade away when you look at coarse data. Once I measured them on the finer data, the model passed both, and it has passed them since.

5. The dashboard

Numbers in a console are not much use, so I built a web page to see the market. It started as a plain chart and grew from there. Green for up, red for down, a candlestick view and a line view. This is the part I spend the most time looking at.

6. Leverage and crashes

I wanted to see a crash, not just a wander. So I added traders who borrow money to trade. When the market turns against them, their losses can wipe out their thin capital, and the system force sells their position. Those forced sales push the price down, which wipes out the next borrower, and so on. That is a cascade, a flash crash.

It did not work at first. The borrowers shared the same big pot of cash as everyone else, so their positions were tiny next to it and they never got into trouble. I gave them their own small capital, and then the crash appeared, sixteen forced sales and a fall of about seventy percent.

7. More traders, a circuit breaker, and a dead end

I added five more trader types to make a fuller market. Mean reversion, breakout, a news trader, a whale working a big order in slices, and a panic herd that sells into falls.

I also added a circuit breaker, the rule real exchanges use to halt trading when the price moves too fast. My first version made things worse. Freezing the price and then reopening created a jump, which set off another halt, a staircase that manufactured fake volatility. I added a cooldown so it could not fire again straight away, and that fixed it.

I also added a tiny delay between a trader deciding and its order arriving, to copy the fact that fast traders act before slow ones. I later took this out, which I explain further down.

8. A live, moving chart

I wanted it to feel like a real trading screen, so I made the chart stream one new bar every minute and keep going without stopping, with a clock ticking. One problem took a while to work out. In the preview the animation kept freezing. It turned out the standard animation method was being paused in the background. I switched to a plain timer and it ran smoothly.

9. Timeframes, growth, and testing strategies

I added one, five, fifteen and thirty minute views, and a small panel showing the change over each of those windows. Then I added a way to test trading strategies. You pick a rule, a moving average cross, momentum, mean reversion, breakout, buy the dip, or buy and hold, tune its settings, and see how it would have done against simply buying and holding.

I tried to go further and let a strategy trade as its own agent, competing with the others. This did not work, and the reason is interesting. The market is chaotic. Even a small order nudges the order book, the momentum traders amplify it, and the whole path turns into a different market. So there is no single fixed market to drop a strategy into and measure cleanly. That is a real feature of this kind of model, not a fault in mine. I kept the simpler and sound approach, testing the rule on the market's price with realistic costs.

10. Real time and looking back

The chart was running too fast, so I set it to real time, one market minute per real minute, with faster options if I want them. I also added the ability to drag the chart back to look at older bars, which I need in order to study patterns, with a button to jump back to the present. I changed the restart button too, so that it brings me back to the current time rather than starting the whole thing over.

11. Making it simple

By this point the code had grown into something more professional than my level. I asked for it to be rebuilt simply, so that it matches my skill and I can read it. The whole engine is now one file, meant to be read from top to bottom, with plain comments.

Two things came out in the rebuild. I removed the tiny order delay, because it made the code harder to follow and barely changed anything. And I removed the half finished trading strategy agent, which was the dead end from before. Removing the delay had one side effect. The leverage crash with the circuit breaker turned out to lean on it, and without it that scenario became unstable. So I changed that scenario to a sharp news shock with the breaker on, which is stable and still shows the breaker doing its job.

12. Real markets

The last big addition was real prices. I wanted to test my strategies against reality, not just the simulation. The app now has a switch between the made up market and real markets. It shows ten real instruments, two each from shares, commodities, natural resources, currencies and crypto, priced in dollars or pounds. Real prices do not fast forward, so you scroll through them. The strategy tester works on the real data too. The prices are a snapshot, downloaded when the page is built, and I can refresh them by running one script. My first data source started blocking scripts, so I switched to another one.

I also tidied the layout so the side panels can be collapsed, and wrote the first version of this log.

13. Following the real market, not a snapshot

Two problems came up once the page was online. The simulation always played out the same way after a restart, because the page replayed one saved run. And the real prices were a snapshot from the day I built the page, so they went stale.

I moved the simulation into the browser, so every visit and every new market starts fresh and random from the first candle. For the real prices I set up a small job on GitHub that downloads fresh prices every morning and rebuilds the page, so the live site follows the market by itself.

14. Starting again with a proper structure

At this point I stepped back. The idea was right, but the way it had been built was not. The strategies were tangled up with the chart, and each market came as one fixed snapshot. I split Agora into four parts, the way real backtesting systems are split. Market data, a strategy engine, a market simulator, and a strategy tester that feeds a research report.

The rule that holds it together is that a strategy should not know whether its prices are real or made up. It just receives bars. So the same rule can be tested on Tesla, on Microsoft, on a made up crash, or on a very volatile market without changing a line.

I widened the data to 29 companies across seven sectors, five indices to compare against, and ten years of daily prices each. The first research report came out of this stage. It also taught me something about my own simulator. The full agent market can run away to absurd prices on some random starts, so it was useless for stress testing. I built a set of simple, controlled synthetic markets for that job instead.

15. Opening up the data on the page

The web page still carried every price inside one big file. I changed it to load each market only when it is picked, which took the page from 589 KB to 55 KB. It gained a picker grouped by sector, the benchmark index drawn next to the strategy, and a small demo of recent 1 hour and 1 minute prices.

16. A backtest that cannot see the future

This was the biggest change of the whole project. My tester had been filling each trade at the same closing price that triggered it. That sounds harmless, but it quietly assumes you could trade at a price that only existed once the day was over. I rebuilt the engine to walk through time one bar at a time. A bar only reaches the strategy once it has closed, and any order waits for the next bar's open.

Prices needed the same care. The data source gives prices already adjusted for splits, which is fine, because every percentage move stays real. But the dividend adjusted price, the one many tutorials use, changes past prices whenever a later dividend is paid, and that leaks the future into the past. So the tester trades on split adjusted prices and pays dividends in as cash on the day they happen. I also moved all times to each exchange's own time zone and started dropping any bar that had not finished yet.

Saying the engine has no look-ahead is easy. I wanted a test that proves it. The test runs a strategy on ten years of prices, then runs it again with the future deleted, at four different dates. The results before each date do not change by a single digit. If the engine were peeking, deleting the future would change the past.

17. Signal, order, fill, and a real account

The next step was to stop strategies deciding everything. Now a strategy only says buy, sell or hold, and why. A sizing rule decides how much. The account applies its limits (the most in one company, how much borrowing, whole shares only). Then an execution model decides the actual price. It adds the bid and ask spread, slippage, the price impact of a large order and commission, and it will not let one order be more than a tenth of a normal day's trading.

The account now works like a broker's. It holds cash and several positions at once, knows the average price paid, splits profit into realised and unrealised, allows short selling, and issues a margin call if a short runs too far against it. The trade log reads the way a real one does. Buy 1,653 shares, expected 60.43, filled at 60.46, fee 49.97, and so on.

18. Testing fairly, and the mistakes it caught

A single backtest proves very little, so the tester now has four methods. A plain backtest, an out-of-sample test that tunes on the early years and judges on the later ones, a walk-forward test that does that year after year, and a stress test on six synthetic markets. Every result also shows what happens with more slippage and with the optimistic same close fills.

The browser needed the same engine, so the page runs a line by line JavaScript copy of the Python one. A parity test runs eight experiments through both and checks that every number matches, so the page and the report cannot disagree.

Questioning results that looked too good caught three mistakes. My first comparison said the walk-forward test beat the plain backtest, which made no sense until I saw the two covered different years

with different settings. The fair comparison is the best settings picked with hindsight on the same years, and against that the honest result was around eight points a year lower. The trade statistics claimed one strategy won 20% of its trades with a profit factor of 0.11 while making 77%, because its best trades were still open at the end of each test year and were not being counted. And a test only three hours long reported a Sharpe ratio of 36, which is what happens when a few hours are stretched into a year. All three are fixed now.

19. Scrolling through real history

The last request was to move around the real market freely, instead of looking at one squashed picture. The chart now holds the whole ten years. You drag to scroll and use the wheel to zoom, and the candles change size to suit, daily when zoomed in, weekly or monthly when zoomed out. A strip underneath maps the whole history, and there is a log scale for long views.

The part I like most is replay. You pick a day in the past and step forward one bar at a time, with everything after it hidden. The tester reruns using only what was known on that day, so you can watch a strategy make its decisions the way you would have faced them. It is the time frontier made visible.

20. What I took out, and why

- The order delay between deciding and trading. Removed to keep the code simple. It barely changed the result.
- The trading strategy agent. Removed because the market is too chaotic to measure a strategy that trades inside it.
- The leverage version of the circuit breaker scenario. Swapped for a news shock, because it became unstable once the order delay was gone.
- My first price data source. It started blocking scripts, so I moved to a different one.
- The one big file of prices inside the page. Replaced by loading each market when it is picked.
- Filling trades at the same close that triggered them. Kept only as an optimistic comparison, because it assumes you could trade at a price you had not seen yet.
- The dividend adjusted price for trading decisions. It rewrites the past with dividends that were paid later.
- The horizon buttons that cut the data down. Replaced by zooming, which keeps the whole history in view, and a separate test period.
- The agent market as the stress test. Replaced by controlled synthetic markets, because the agent market can run away on some random starts.

21. Where it stands, and honest notes

The simulator still works and passes both realism checks. Around it there is now a proper research tool. It has 29 companies and 6 indices refreshed daily, a backtest engine tested for look-ahead, a realistic execution model and account, four ways to test a rule, and a web page that runs the same engine and lets you scroll through or replay ten years of prices.

Some honest limits. The twelve companies in the research were picked knowing they did well, which flatters buy and hold. The costs are modelled, not measured from real order books. Intraday data only

goes back a week at one minute. And the main answer the research gave, that none of the simple rules beat holding these shares once tested fairly, comes from one ten year period that was mostly a rising market.

I am pleased with it. More than the app itself, I learned how a market can be built out of simple rules, why it is hard to keep such a thing stable, and why testing a trading strategy is not as simple as it first sounds.